Kubernetes: внутреннее устройство

Архитектура, компоненты и принципы работы кластера

Докладчики: Пашкевская Дана, Часовских Кристиан

План доклада

1. Проблема оркестрации
2. Определение и объекты
3. Архитектура кластера
4. Control Plane
5. Worker Node и сеть
6. Сквозной сценарий
7. Самовосстановление
8. Преимущества и ограничения
9. Практика: демонстрации

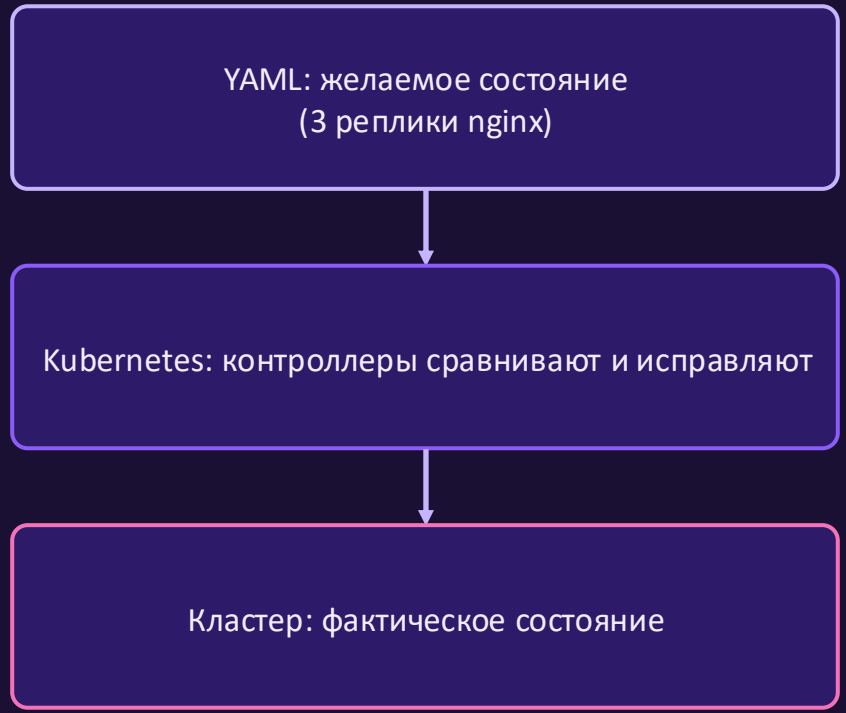
Проблема: зачем нужна оркестрация?

- Контейнер решает упаковку приложения, но не эксплуатацию
- Сотни контейнеров на десятках серверов: кто запускает и следит?
- Нужны: автоперезапуск, масштабирование, балансировка, обновления без простоя
- Borg (Google) → Kubernetes (2014) → CNCF



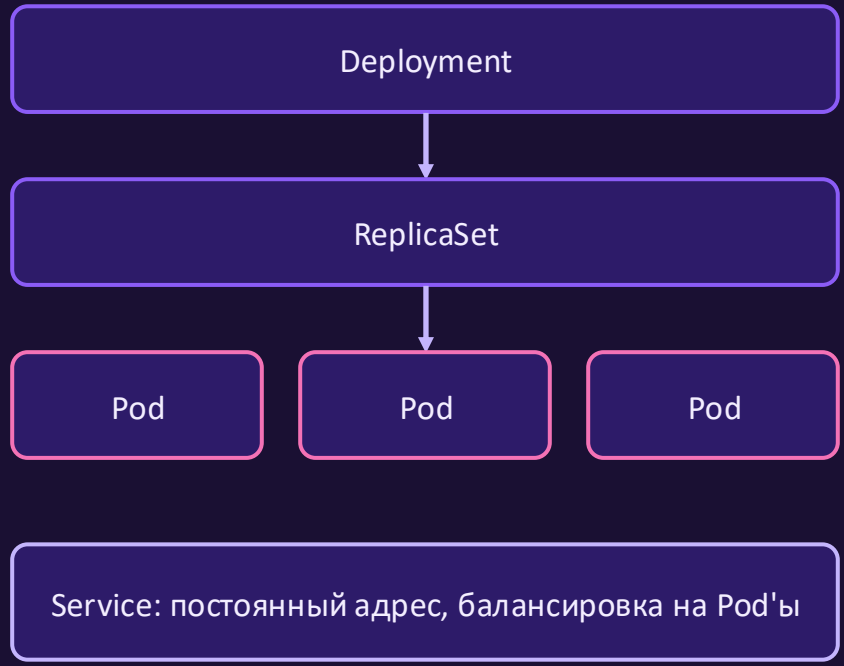
Что такое Kubernetes?

- Платформа оркестрации контейнеров с открытым исходным кодом
- Декларативная модель: описываем желаемое состояние, система сама приводит к нему фактическое
- Работает постоянно, а не один раз
- Аналоги: Docker Swarm, HashiCorp Nomad

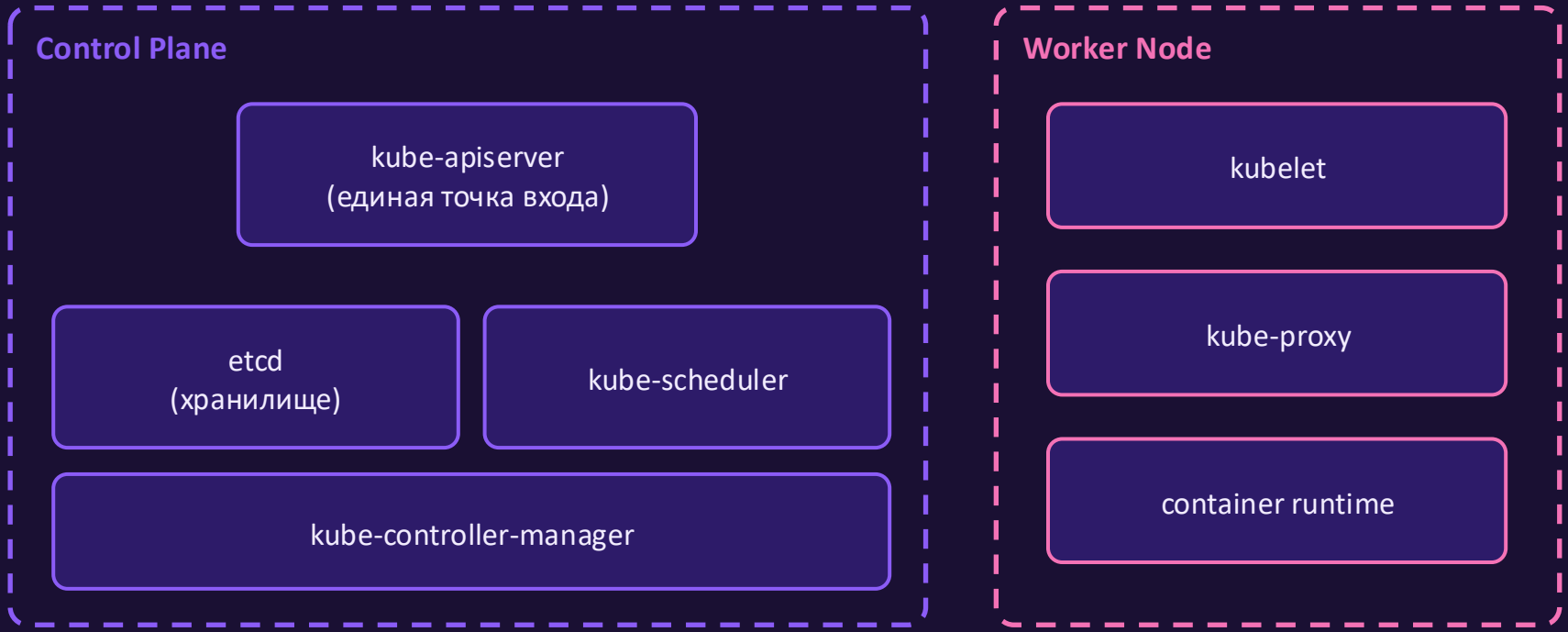


Основные объекты

- Pod: контейнеры с общей сетью и томами
- ReplicaSet: поддерживает число Pod'ов
- Deployment: управляет ReplicaSet, обновления и откат
- Service: постоянный адрес и балансировка
- ConfigMap / Secret, Namespace



Архитектура кластера



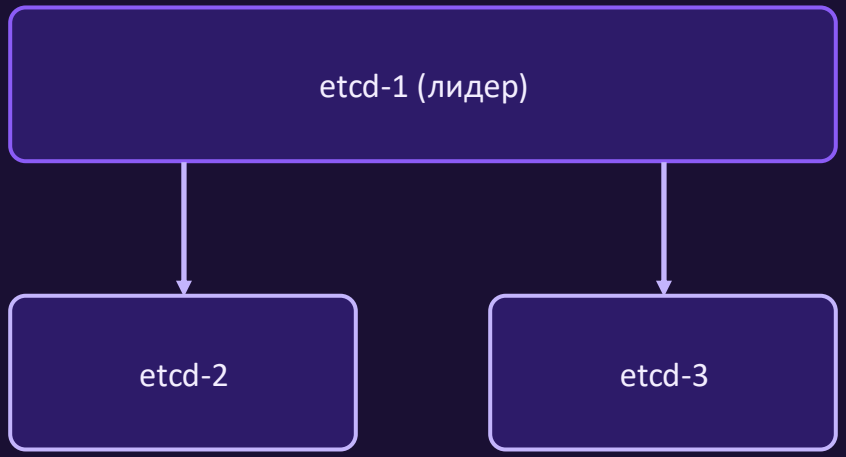
kube-apiserver

- Единая точка входа в кластер (REST API)
- Каждый запрос проходит проверки, прежде чем попасть в etcd
- Единственный компонент, напрямую работающий с etcd
- Watch: клиенты подписываются на изменения



etcd

- Распределённое key-value хранилище: источник истины
- Ключи вида /registry/pods/demo/...
- Raft: кворум n/2 + 1, узлов нечётное число (3, 5)
- Без копии etcd кластер «забывает» своё состояние



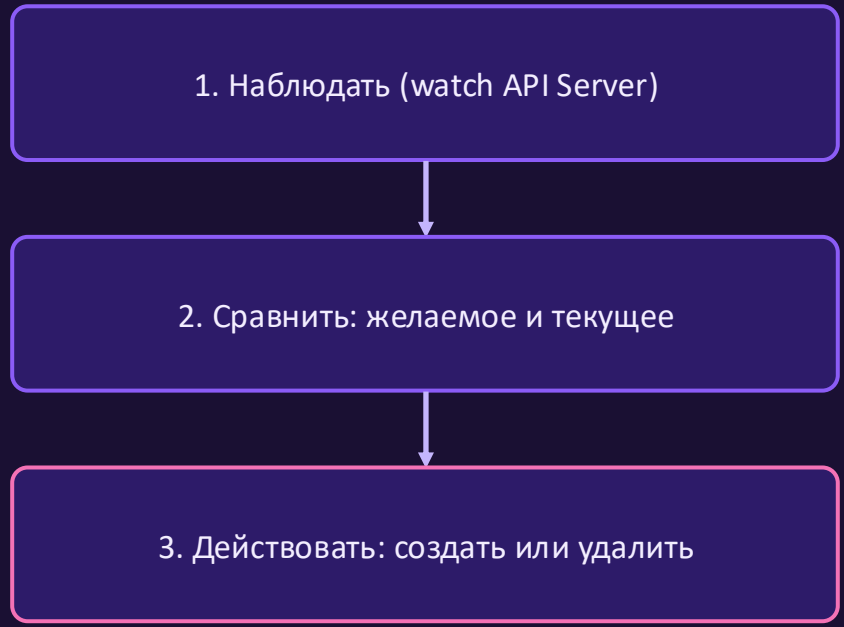
kube-scheduler

- Выбирает узел для нового Pod'а
- Фильтрация: ресурсы, taints, selector'ы, affinity
- Ранжирование: баллы по оставшимся узлам
- Только пишет nodeName, Pod запускает kubelet



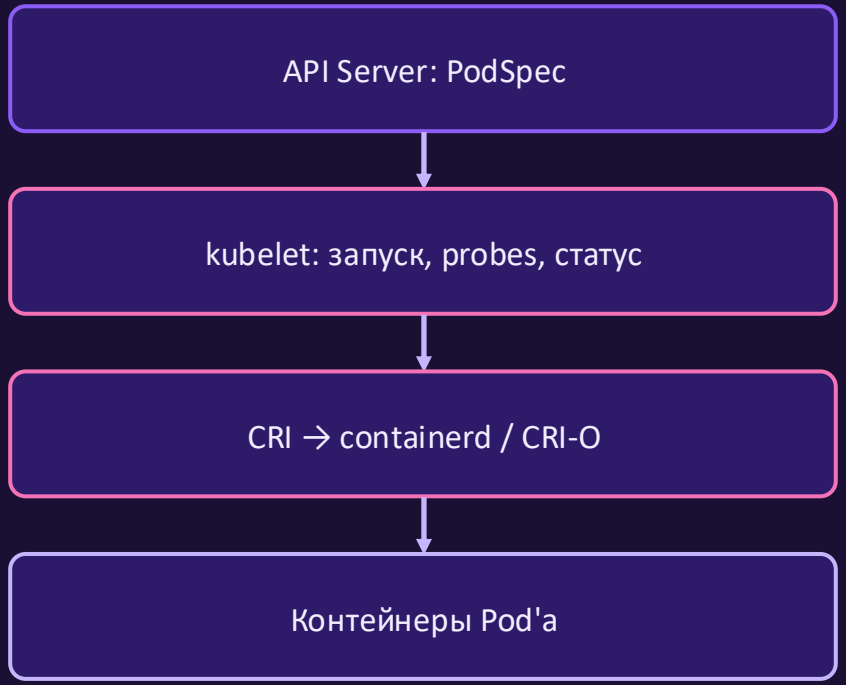
kube-controller-manager

- Контроллеры: Deployment, ReplicaSet, Node, Job и др.
- Цикл согласования работает постоянно
- Пример: нужно 3 Pod'а, есть 2 → создаётся ещё один
- cloud-controller-manager: интеграция с облаком



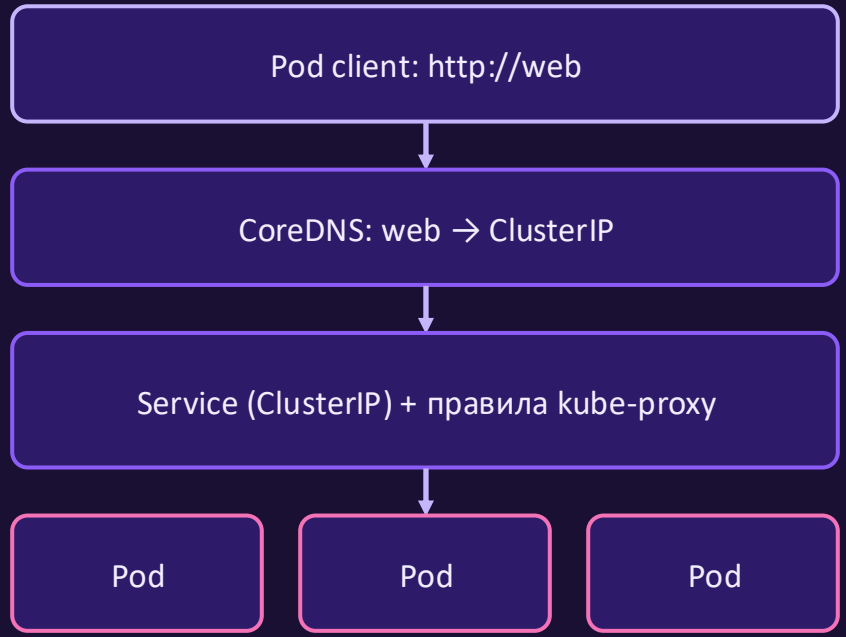
kubelet и container runtime

- kubelet: агент на каждом узле, получает PodSpec от API Server
- Проверки: liveness, readiness, startup probes
- Runtime (containerd, CRI-O) запускает контейнеры через CRI, CNI: сеть, CSI: хранилище
- Dockershim удалён в Kubernetes 1.24

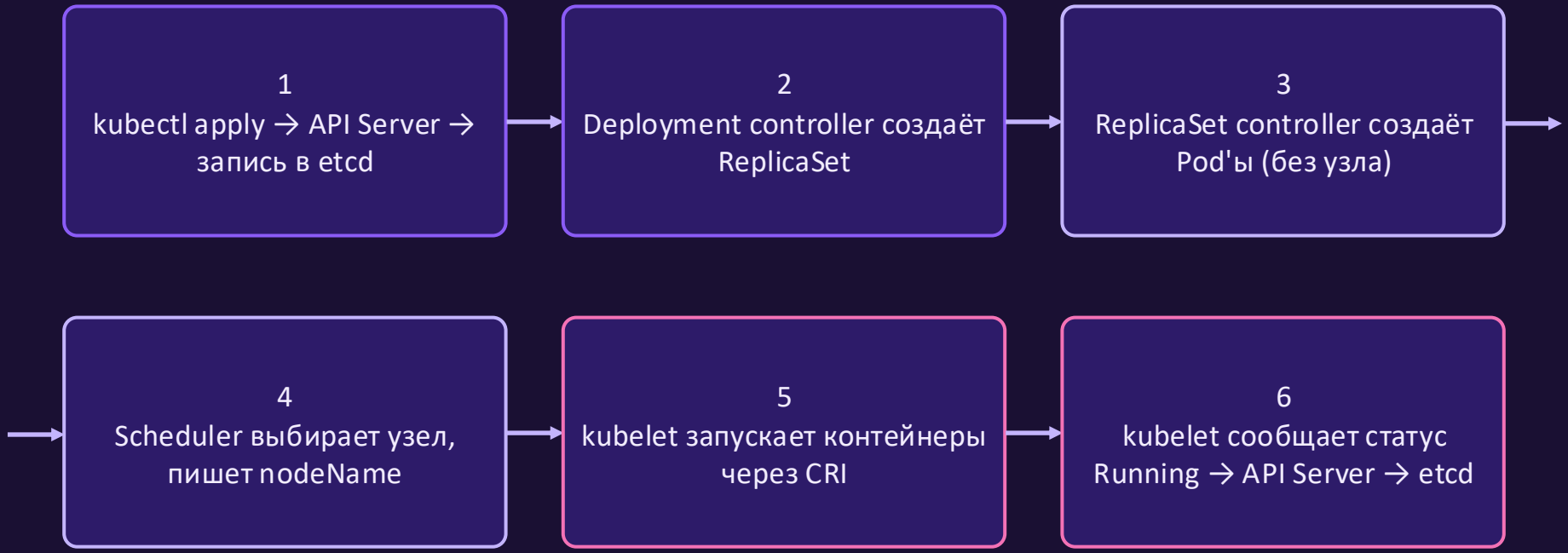


kube-proxy и сеть

- У каждого Pod свой IP, Pod'ы общаются без NAT
- Service даёт постоянный ClusterIP, kube-proxy настраивает iptables / IPVS
- CoreDNS переводит имя сервиса в адрес
- CNI-плагины: Calico, Flannel, Cilium



Сквозной сценарий: создание Deployment

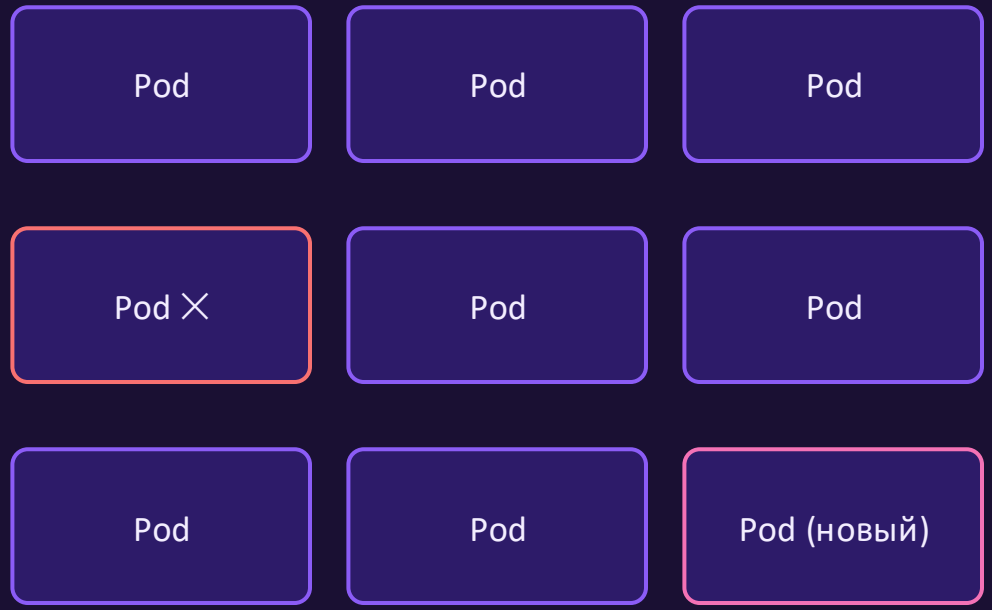


Самовосстановление и масштабирование

Желаемое состояние: 3 Pod

Pod упал или узел недоступен

Контроллер создаёт замену



Преимущества и ограничения

Преимущества

- Отказоустойчивость и самовосстановление
- Переносимость между облаками
- Расширяемость: CRD, операторы
- Большая экосистема

Ограничения

- Высокая сложность, порог входа
- Накладные расходы кластера
- Избыточен для небольших проектов
- Безопасность и сеть требуют настройки

Практическая часть

Стенд: кластер kind (1 control-plane, 2 worker) в Docker, управление через kubectl

Демо 1. Путь запроса	Применение манифеста Deployment, просмотр связки Deployment → ReplicaSet → Pod и размещения Pod'ов на worker-узлах.
Демо 2. Самовосстановление	Ручное удаление Pod'а и его автоматическое восстановление, масштабирование с трёх до пяти реплик и обратно.
Демо 3. Service и обновление	Service и EndpointSlice, доступ по DNS-имени web, запросы в логах трёх Pod'ов, обновление nginx 1.25 → 1.27 и откат.
Демо 4. Внутренние компоненты кластера	Системные Pod'ы kube-system, манифесты static Pod'ов, проверка готовности API Server (/readyz) и здоровья etcd.

Спасибо за внимание!